IT3280 – THỰC HÀNH KIẾN TRÚC MÁY TÍNH

BÁO CÁO THỰC HÀNH

BÀI 4: Các lệnh số học và logic.

Họ và tên: Nguyễn Hồng Khôi

MSSV: 202416956

Assignment 1:

Chương trình:

.text

```
        li s1,9999999999
li s2,9999999999
# Thuật toán xác định tràn số
        li t0, 0                    # Mặc định không có tràn số
        add s3, s1, s2              # s3 = s1 + s2
        xor t1, s1, s2              # Kiểm tra s1 với s2 có cùng dấu
        blt t1, zero, EXIT          # Nếu t1 là số âm, s1 và s2 khác dấu
        blt s1, zero, NEGATIVE      # Kiểm tra s1 và s2 là số âm hay không âm
bge s3, s1, EXIT    # s1 không âm, kiểm tra s3 nhỏ hơn s1 không
# Nếu s3 >= s1, không tràn số
        j OVERFLOW
NEGATIVE:
        bge s1, s3, EXIT # s1 âm, kiểm tra s3 có lớn hơn s1 không
# Nếu s1 >= s3, không tràn số
OVERFLOW:
        li t0, 1 # The result is overflow
EXIT:
```

Lệnh sử dụng:

- Lệnh xor rd, rs1, rs2 thực hiện phép toán XOR từng bit của 2 thanh ghi rs1 và rs2, kết quả được ghi vào thanh ghi rd.

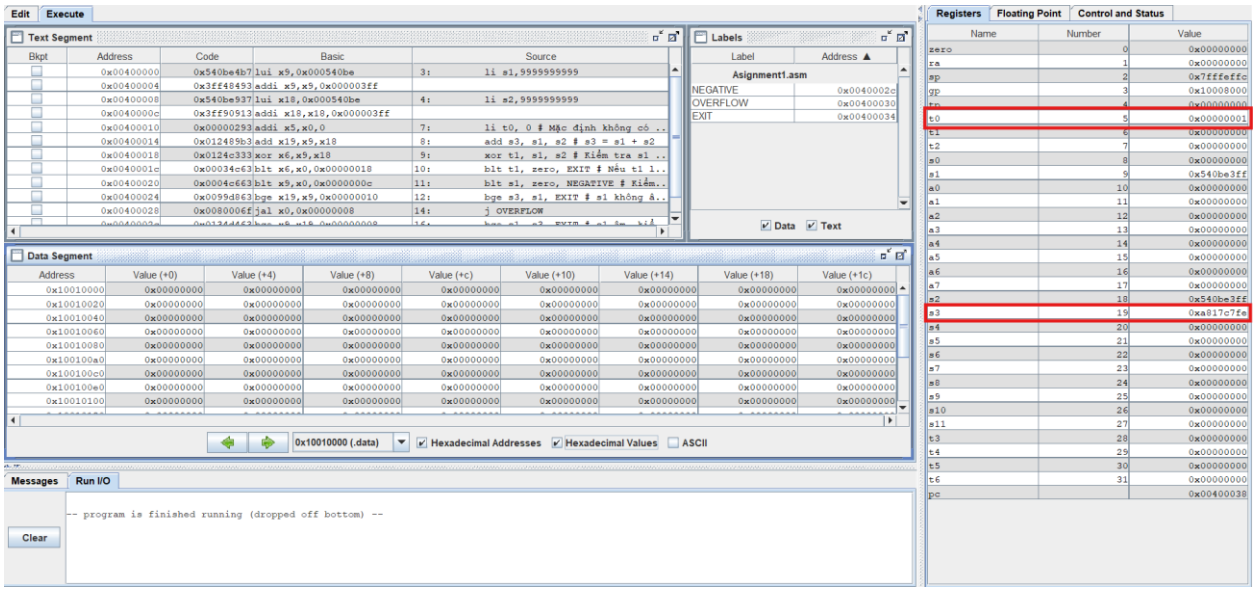
Trong chương trình của bài này, lệnh xor được sử dụng để kiểm tra dấu của s1 và s2.

Các số 32-bit có dấu đều dùng bit đầu tiên để thể hiện dấu của số đó. Số âm có bit đầu là 1, số dương là 0. Do vậy có thể phát hiện 2 số có trái dấu không bằng cách XOR từng bit của 2 số. Nếu kết quả của phép toán là số âm. Tức bit đầu là 1,

chứng tỏ 2 số trái dấu. Khi thực hiện phép cộng, nếu 2 số trái dấu thì việc tràn số không thể xảy ra. TH1 : 2 số dương tràn số

```
li s1,9999999999
li s2,9999999999
```

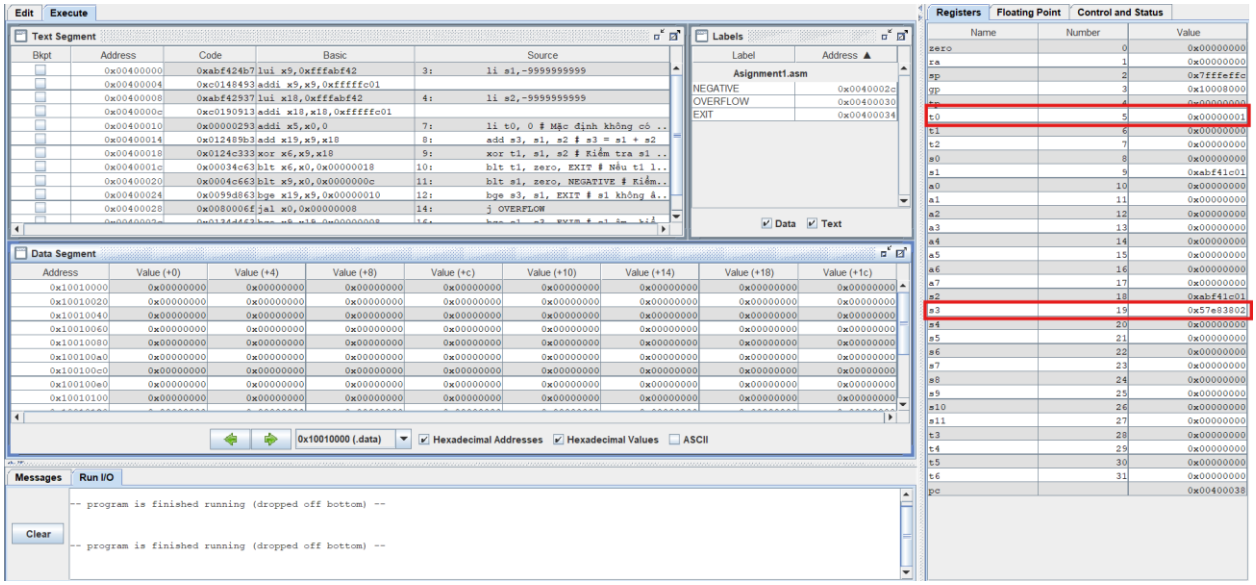
Kết quả:



TH2: 2 số âm tràn số

```
li s1,-9999999999
li s2,-9999999999
```

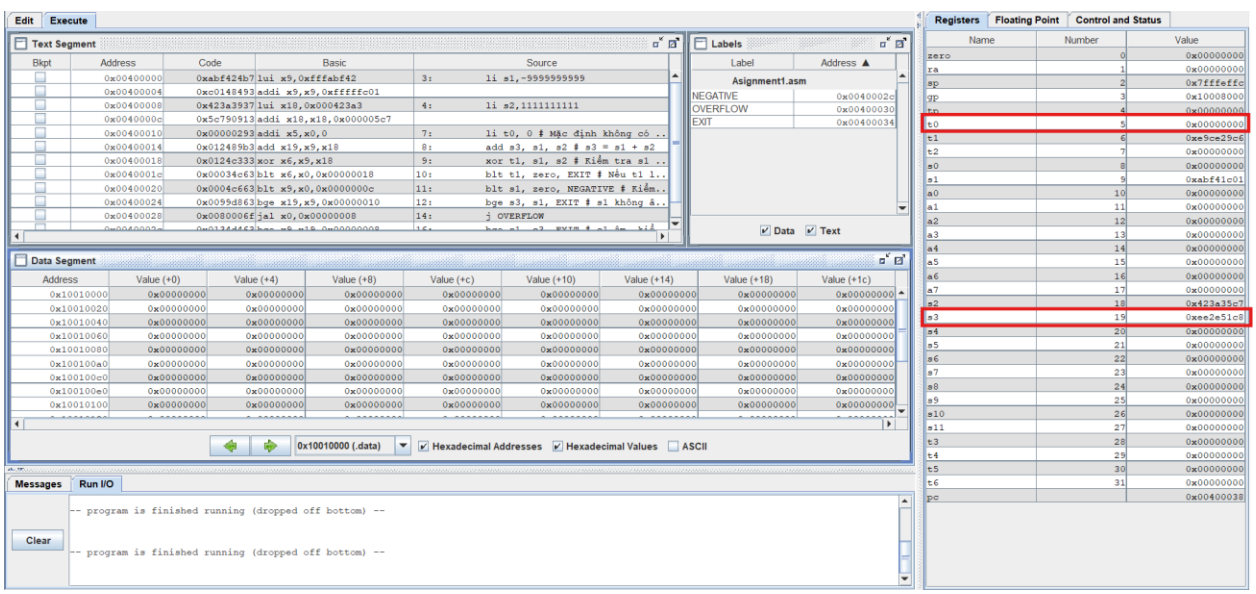
Kết quả



TH3: không tràn số

```
li s1,-9999999999
li s2,1111111111
```

Kết quả:



Assignment 2:

Chương trình:

.text li s0, 0x12345678 srli s1, s0, 24

#Trich xuat MSB li t1, 0xFFFFFF00

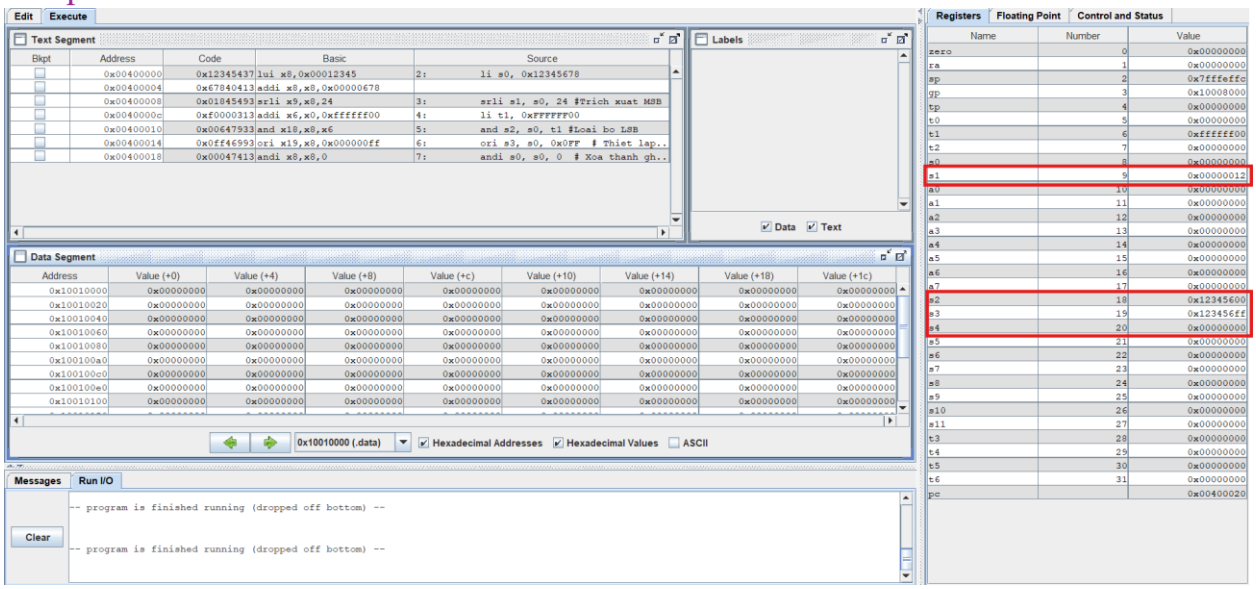
and s2, s0, t1 #Loai bo LSB ori

s3, s0, 0x0FF # Thiet lap LSB andi

s0, s0, 0 # Xoa thanh ghi s0

- Trích xuất MSB của thanh ghi s0: Ta có thể sử dụng lệnh srli dịch bit sang trái 6 byte = 24 bit để đưa 2 byte MSB về cuối.
- Xoá LSB của thanh ghi s0: Để xoá giá trị của 1 bit nào đó có thể AND nó với 0, để xoá 2 byte = 8 bit cuối, thì cần AND với 0xFFFFFF00, vì số này vượt quá 12bit có thể biểu diễn được nên ta sẽ dùng câu lệnh add rd, rs1, rs2.
- Thiết lập LSB của thanh ghi s0: Để đặt giá trị của 1 bit thành 1, ta có thể OR nó với 1, do đó ta cần thực hiện phép OR s0 với 0x0FF.
- Xoá thanh ghi s0: Nhờ tính chất của phép XOR, trong phép toán nếu 2 số hạng giống nhau cho ra kết quả 0, khác nhau thì ra 1. Vì vậy để xoá thanh ghi s0 chỉ cần làm phép toán s0 XOR s0.

Kết quả:



Assignment 3:

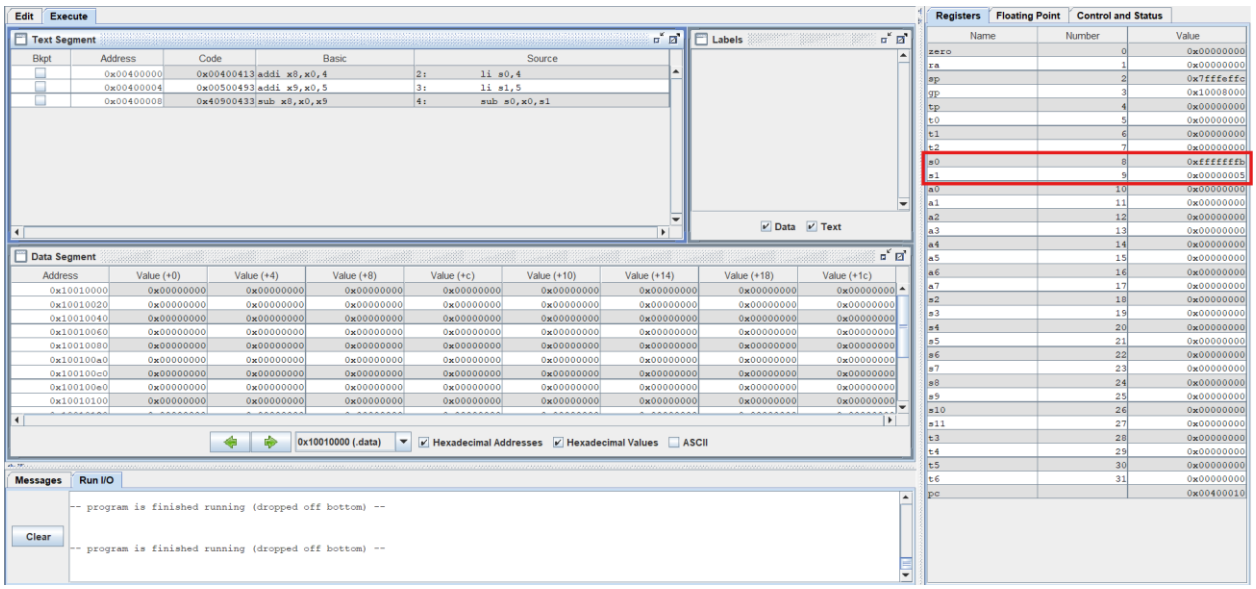
a, Chương trình:

.text li s0,4

li s1,5 sub

s0,x0,s1

Kết quả:



Lệnh neg rd, rs1 thực hiện việc đảo dấu của rs1 và ghi lại vào thanh ghi rd.

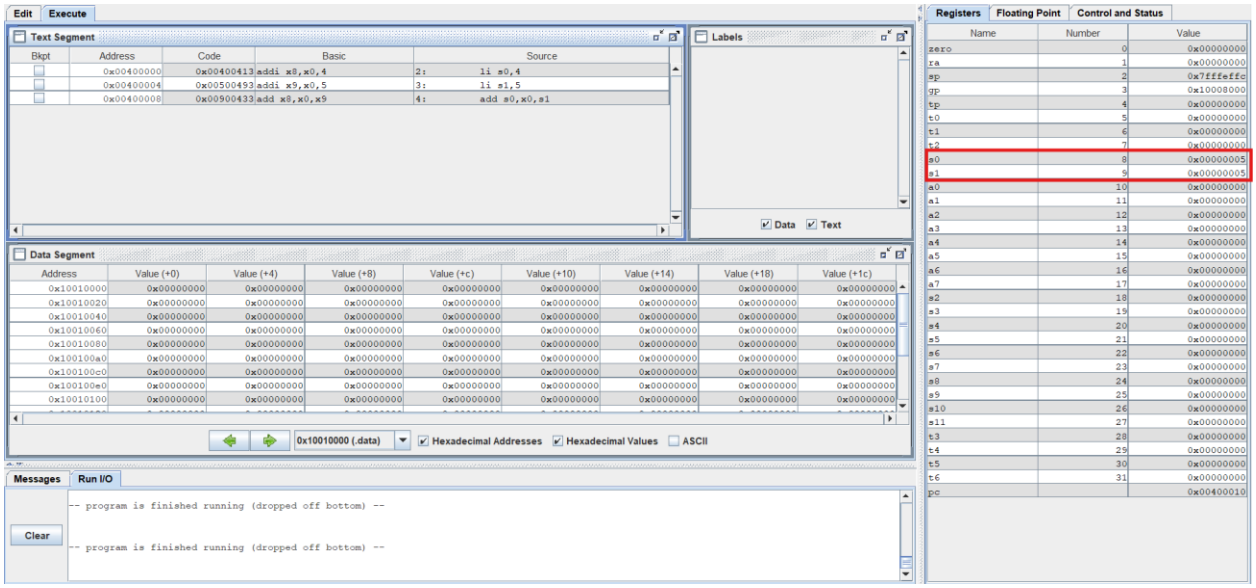
b, Chương trình:

.text li s0,4

li s1,5 add

s0,x0,s1 Kết

quả



Lệnh mv rd, rs1 thực hiện sao chép rs1 và ghi vào thanh ghi rd.

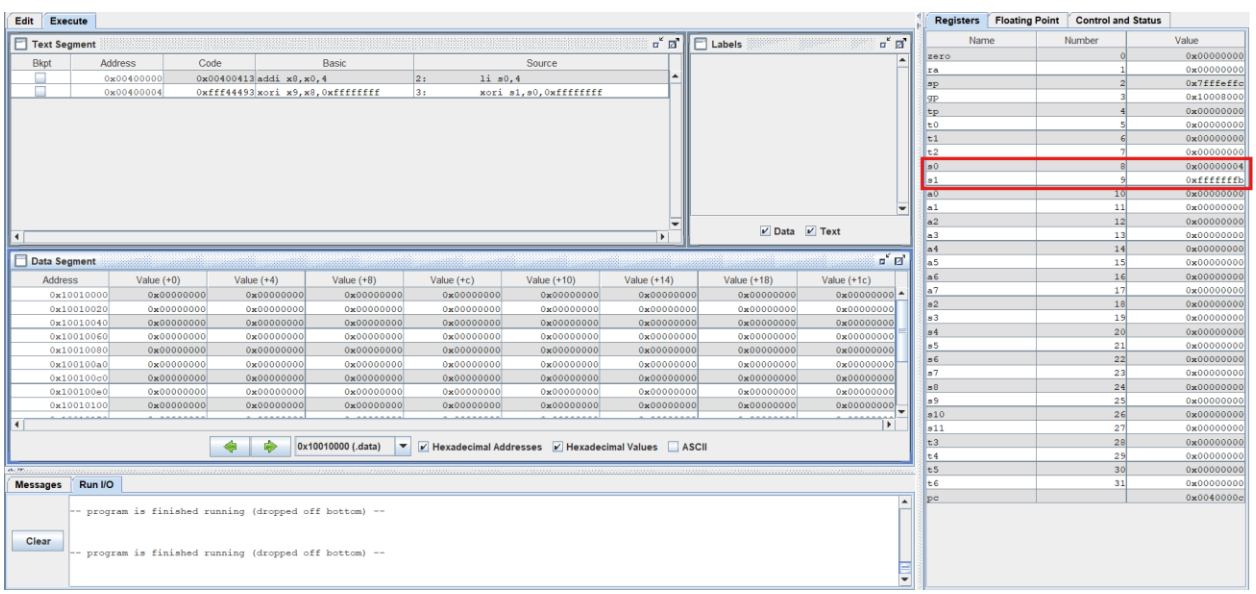
c, Chương trình:

.text li

s0,4

xori s1,s0,0xffffffff Kết

quả:



Lệnh not rd, rs1 lật tất cả các bit của rs1 và ghi vào thanh ghi rd

d, Chương trình:

.text li

s0, 5

li s1, 4

li t0, 0

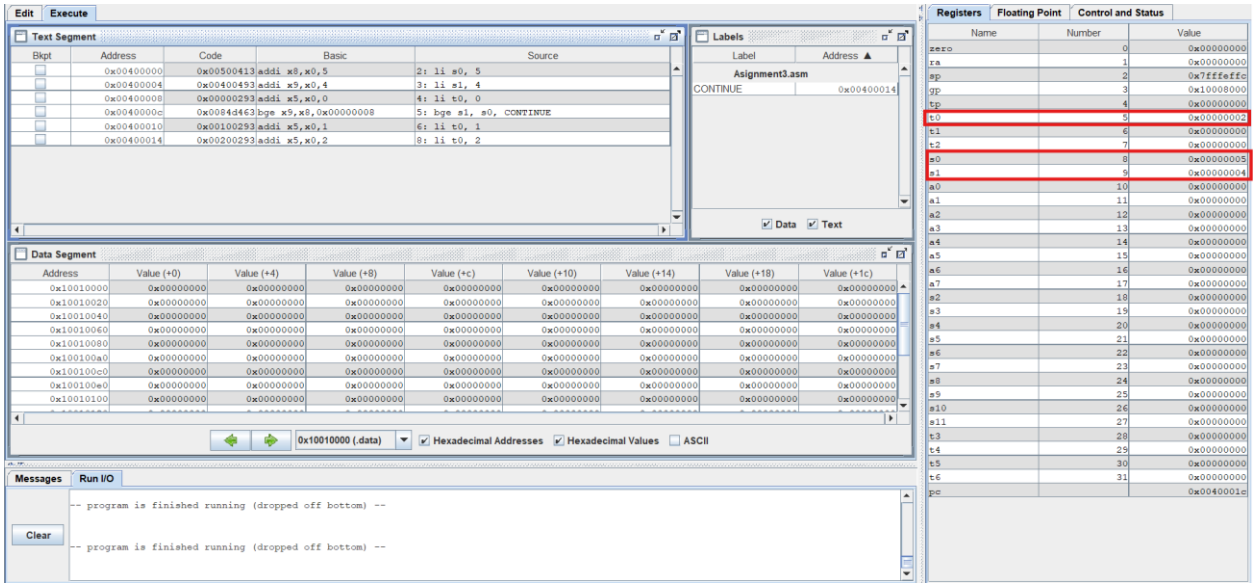
bge s1, s0, CONTINUE

li t0, 1

CONTINUE: li

t0, 2

Kết quả:



Lệnh ble rs1, rs2, label thực hiện nhảy đến nhãn label nếu rs1 ≤ rs2

Assignment 4:

Chương trình:

.text li s1,-

9999999999 li

s2,-9999999999 li

t0, 0 # Mặc định

không có tràn số

add s3, s1, s2 # s3

= s1 + s2 xor t1,

s1, s2 # Kiểm tra

s1 với s2 có cùng

dấu blt t1, zero,

EXIT # Nếu t1 là

số âm, s1 và s2

khác dấu xor

t2,s3,s1 blt

t2,zero,

OVERFLOW j

EXIT

OVERFLOW:

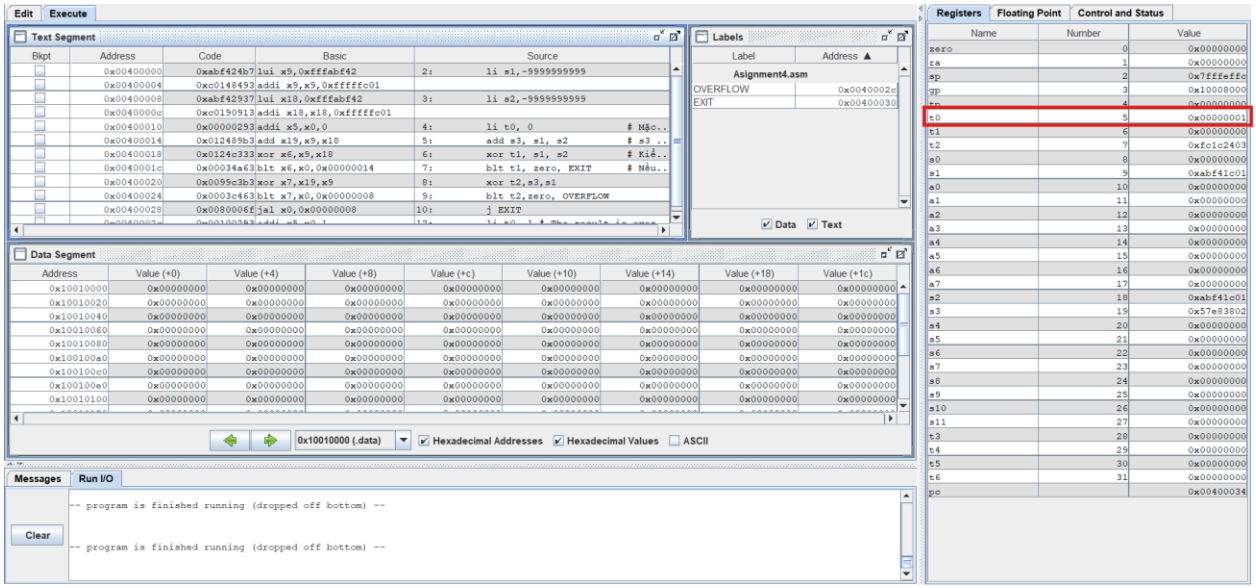
li t0, 1 # The result is overflow

EXIT:

TH1: 2 số cùng dấu tràn số

```
li s1,-9999999999
li s2,-9999999999
```

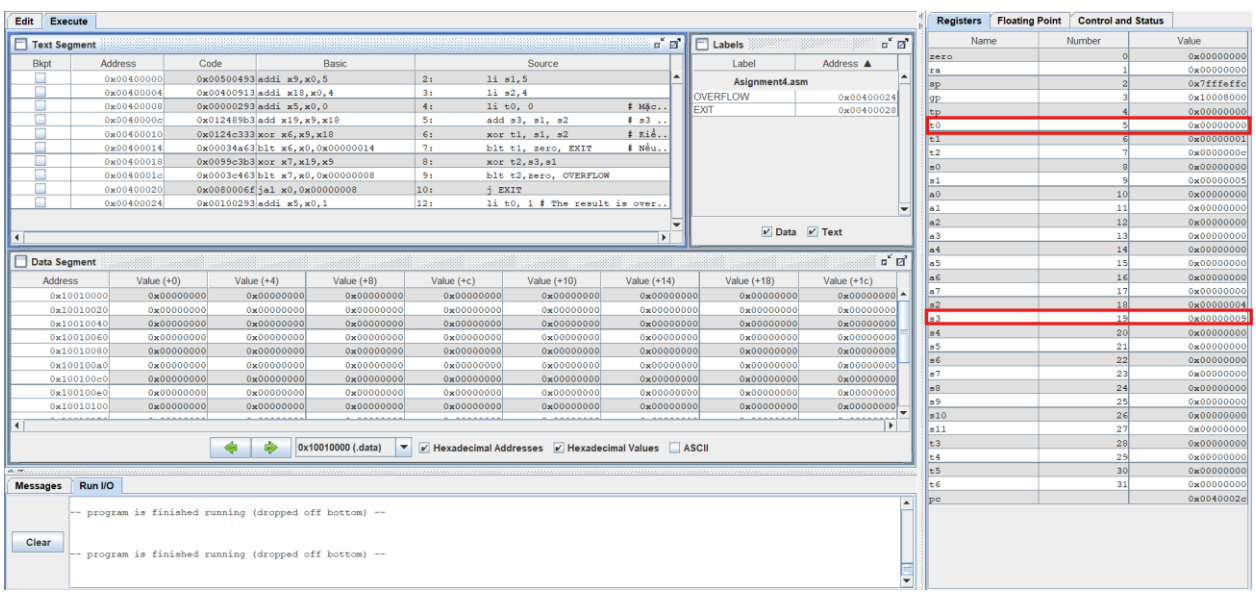
Kết quả:



TH2: 2 số cùng dấu không tràn số

```
li s1,5
li s2,4
```

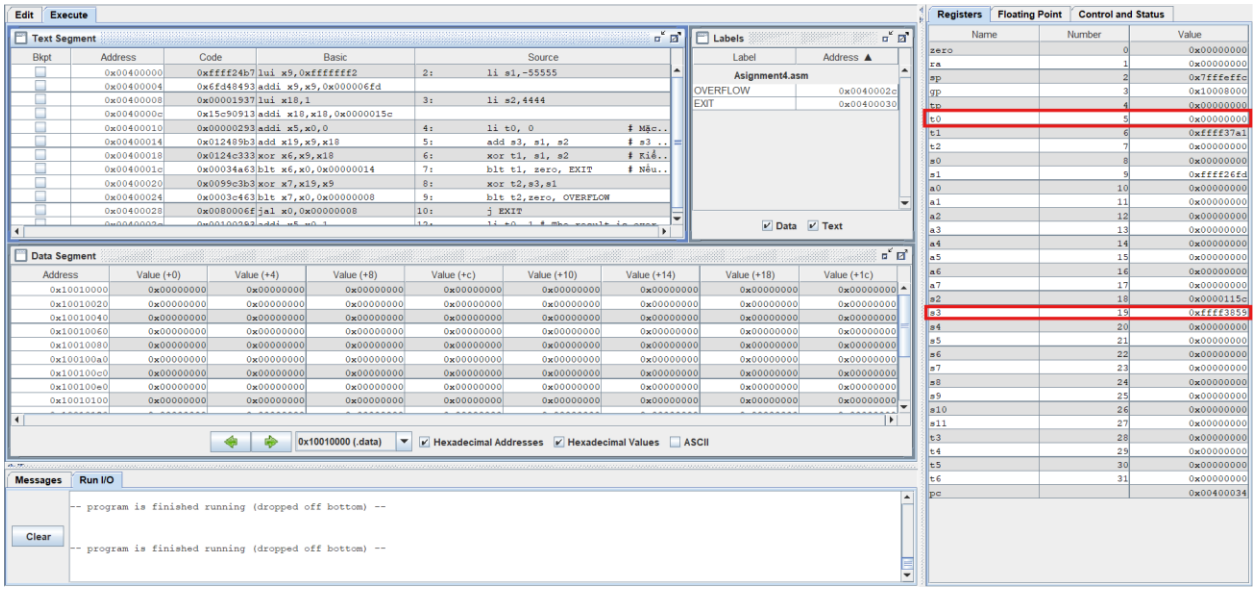
Kết quả:



TH3: 2 số khác dấu

```
li s1,-55555
li s2,4444
```

Kết quả:

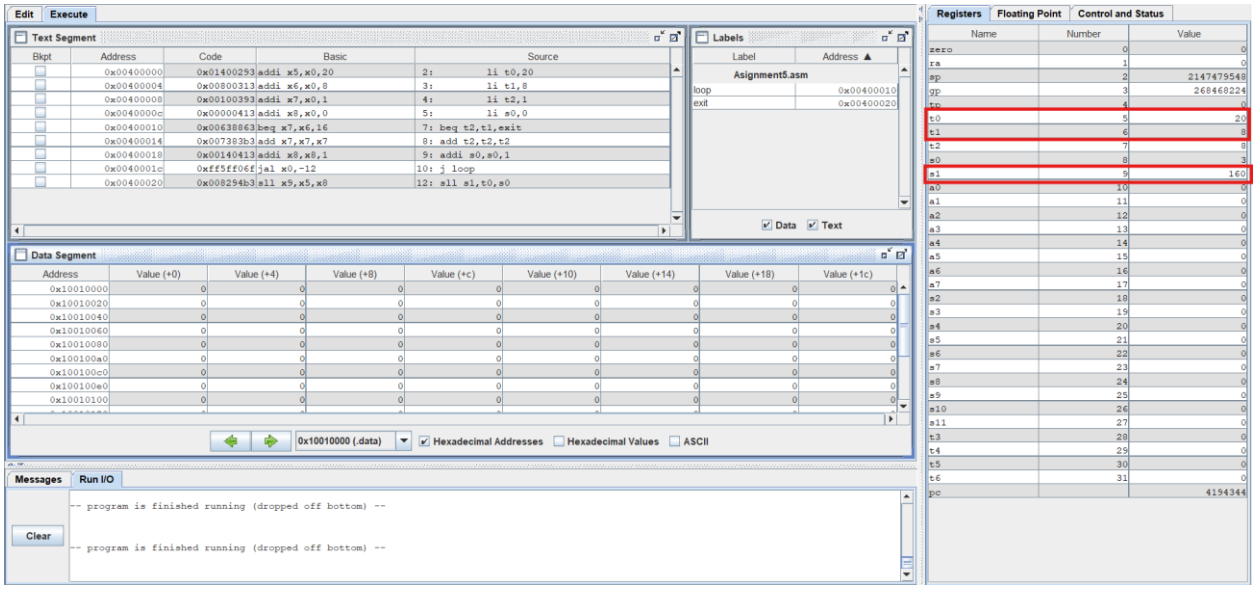


Assignment 5:

Chương trình:

```
.text

        li t0,20
li t1,8         li
t2,1    li s0,0
loop:                   # Vòng lặp đếm số mũ
beq t2,t1,exit          # t2=t1 nhảy đến exit

add t2,t2,t2            # t2=t2*2

addi s0,s0,1            # s0 là số mũ của 2, mỗi lần lặp tăng 1
j loop exit:

sll s1,t0,s0
```

Kết quả:



Edit | Execute

Text Segment

Bkpt	Address	Code	Basic	Source
	0x00400000	0x01400293	addi x5,x0,20	2: li t0,20
	0x00400004	0x00800313	addi x6,x0,8	3: li t1,8
	0x00400008	0x00100393	addi x7,x0,1	4: li t2,1
	0x0040000c	0x00000413	addi x8,x0,0	5: li s0,0
	0x00400010	0x00638863	beq x7,x6,16	7: beq t2,t1,exit
	0x00400014	0x007383b3	add x7,x7,x7	8: add t2,t2,t2
	0x00400018	0x00140413	addi x8,x8,1	9: addi s0,s0,1
	0x0040001c	0xff5ff06f	jal x0,-12	10: j loop
	0x00400020	0x008294b3	sll x9,x5,x8	12: sll s1,t0,s0

Labels

Label	Address ▲
Asignment5.asm	
loop	0x00400010
exit	0x00400020

☑ Data ☑ Text

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0	0	0	0	0	0	0	0
0x10010020	0	0	0	0	0	0	0	0
0x10010040	0	0	0	0	0	0	0	0
0x10010060	0	0	0	0	0	0	0	0
0x10010080	0	0	0	0	0	0	0	0
0x100100a0	0	0	0	0	0	0	0	0
0x100100c0	0	0	0	0	0	0	0	0
0x100100e0	0	0	0	0	0	0	0	0
0x10010100	0	0	0	0	0	0	0	0

0x10010000 (.data) ☑ Hexadecimal Addresses ☐ Hexadecimal Values ☐ ASCII

Messages | Run I/O

Clear

```
-- program is finished running (dropped off bottom) --

-- program is finished running (dropped off bottom) --
```

Registers | Floating Point | Control and Status

Name	Number	Value
zero	0	0
ra	1	0
sp	2	2147479548
gp	3	268468224
tp	4	0
t0	5	20
t1	6	8
t2	7	8
s0	8	3
s1	9	160
a0	10	0
a1	11	0
a2	12	0
a3	13	0
a4	14	0
a5	15	0
a6	16	0
a7	17	0
s2	18	0
s3	19	0
s4	20	0
s5	21	0
s6	22	0
s7	23	0
s8	24	0
s9	25	0
s10	26	0
s11	27	0
t3	28	0
t4	29	0
t5	30	0
t6	31	0
pc		4194344